

Classes IOS – MVC

X-Code – Swift

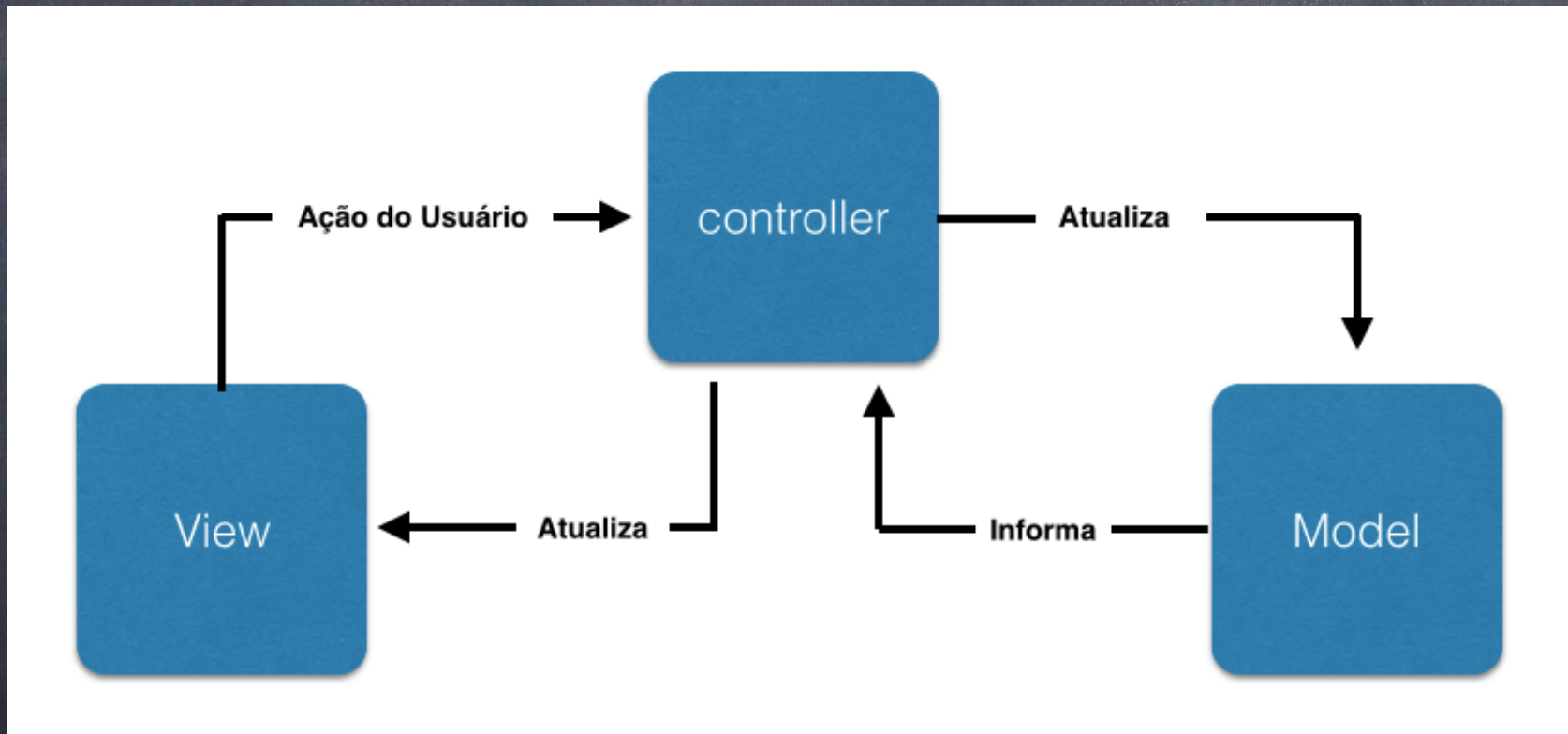
Prof. Agesandro Scarpioni

agesandro@fiap.com.br

MVC – Model View Controller

- MVC é um padrão de projeto, padrão de projeto ou Design Patterns, em uma forma simples de explicar, são soluções para problemas que alguém um dia teve e resolveu aplicando um modelo que foi documentado e que podemos adaptar em nossa solução.
- O MVC é uma forma de estruturar o projeto/aplicação de forma que a interface de interação (View) seja separada do controle da informação (Models), essa separação é intermediada por outra camada controladora (Controllers).

MVC – Model View Controller



Criar o Projeto

- Em um projeto do tipo App crie um projeto chamado: Exemplo MVC Swift.

Choose a template for your new project:

Multiplatform **iOS** macOS watchOS tvOS Other Filter

Application

Choose options for your new project:

 **App**

 iMessage App

Framework & Library

 Framework

Cancel

Product Name:

Team:

Organization Identifier:

Bundle Identifier:

Interface:

Language:

☐ Use Core Data

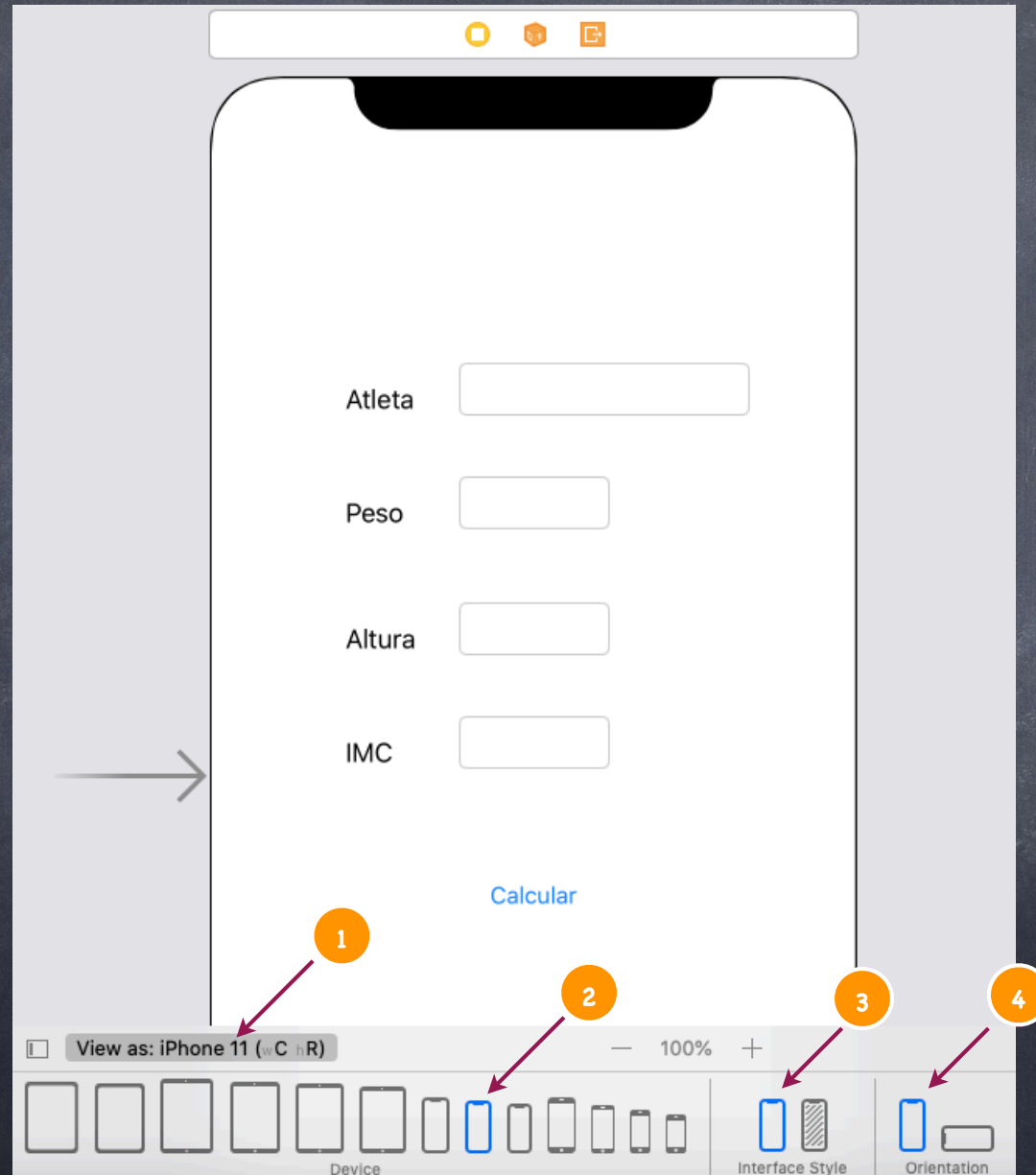
☐ Host in CloudKit

☐ Include Tests

Cancel Previous Next

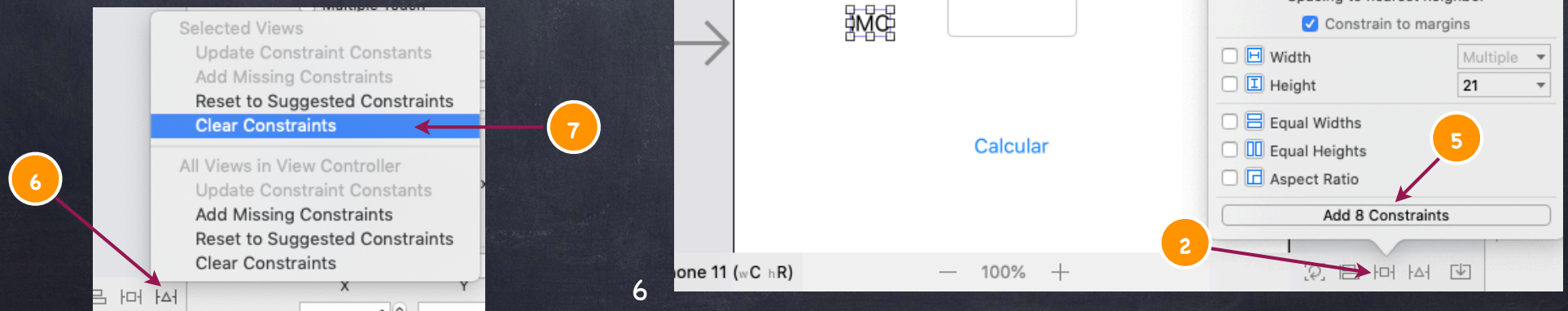
Desenhando a interface

- Crie no Storyboard a seguinte View com label's, Text's e um Button, utilize para seu desenho (1) o iPhone 11 (2), Light Style (3), orientação Portrait/Retrato (4).



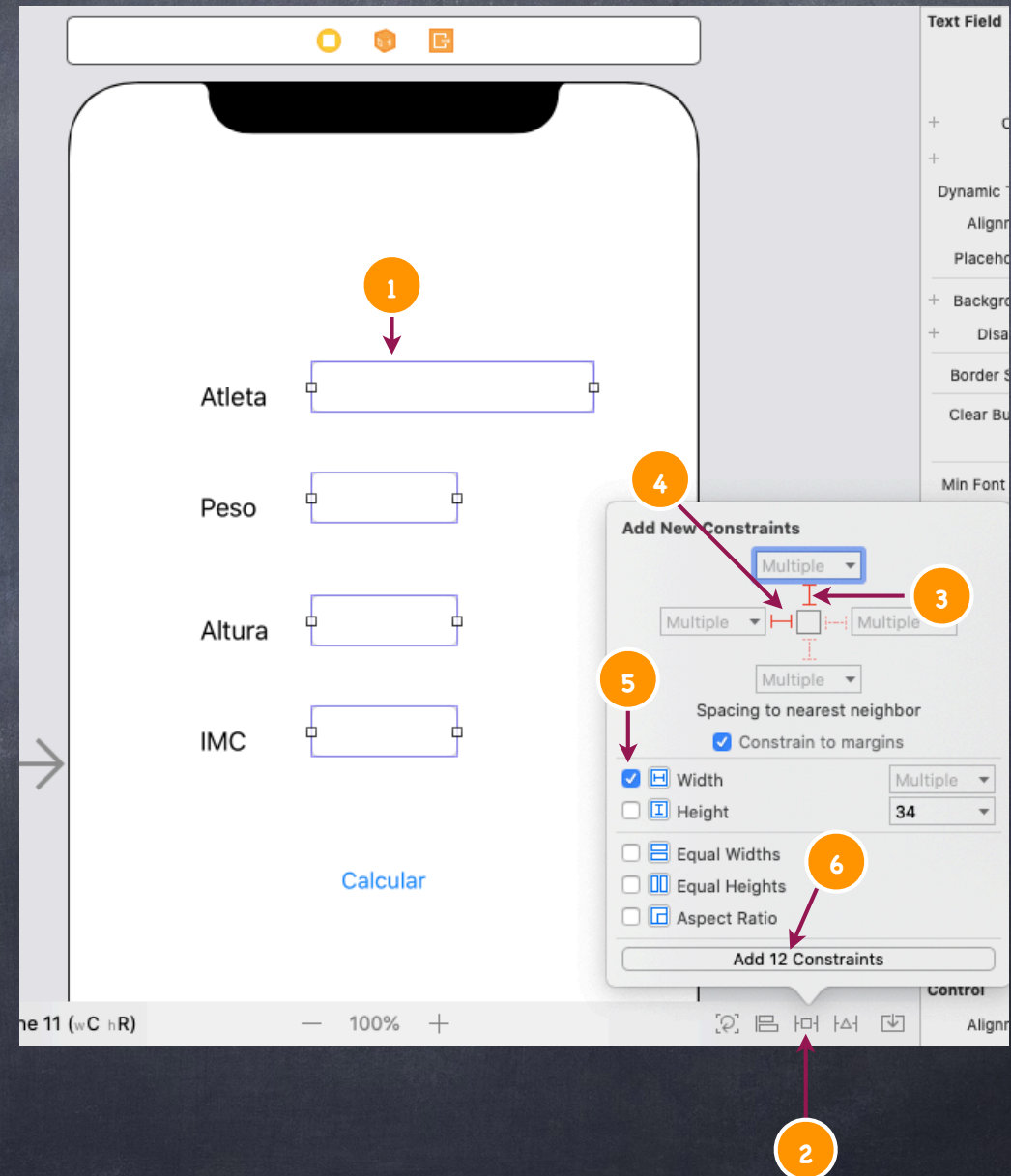
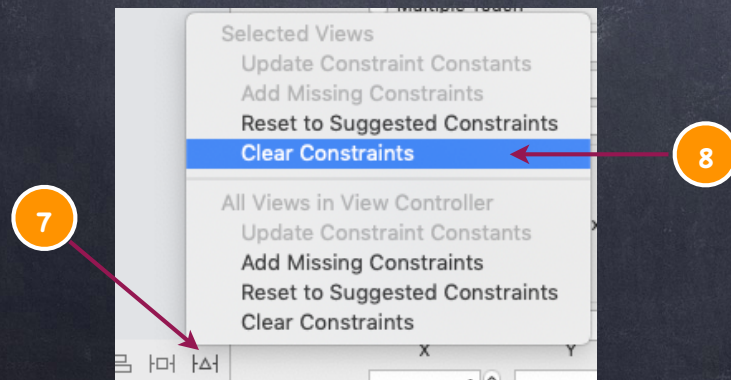
Auto Layout

- Na Aula 09 aprenderemos mais detalhes sobre Auto Layout, essa técnica serve para que seu App rode em qualquer tamanho de aparelho, no entanto, vamos mostrar neste pequeno App como fazer esse controle, como inserir as chamadas "Constraints".
- Siga o passo a passo: Selecione todos os Labels (1), clique em Add New Constraints (2), defina as constraints de topo (3) e esquerda (4), depois cliquem em Add 8 constraints (5).
- Se você errar não se preocupe, com os Labels selecionados, clique em Resolve Auto Layout Issues (6), depois clique em Clear Constraints (7) e repita o passo anterior.



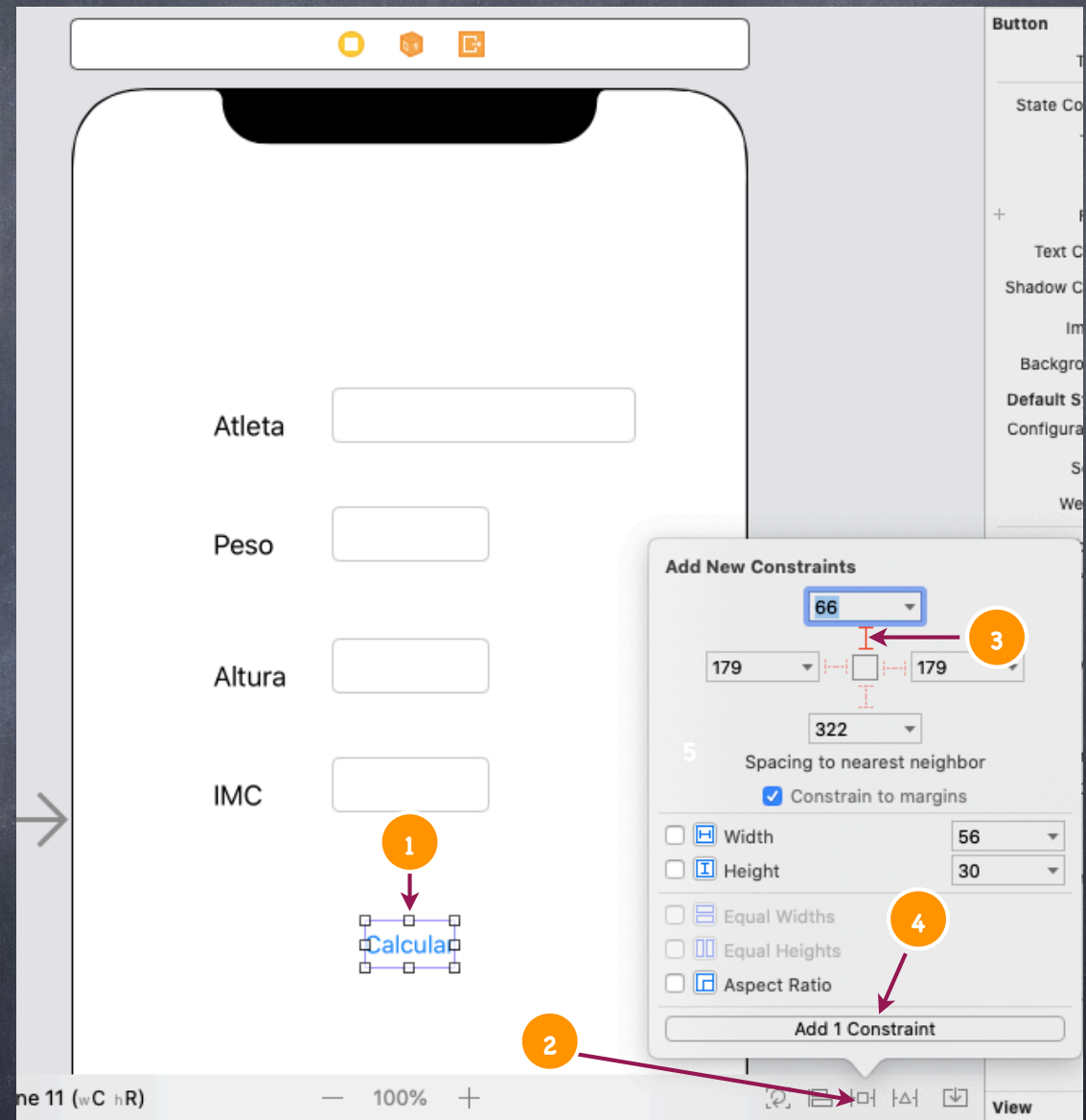
Auto Layout

- Selecione todos os Text's (1), clique em Add New Constraints (2), defina as constraints de topo (3) esquerda (4), Width (5) depois cliquem em Add 12 constraints (6). O Width foi marcado para que a largura fique fixa conforme foi desenhada.
- Repetindo, se você errar não se preocupe, com os Text's selecionados, clique em Resolve Auto Layout Issues (7), depois clique em Clear Constrains (8) e repita o passo anterior.



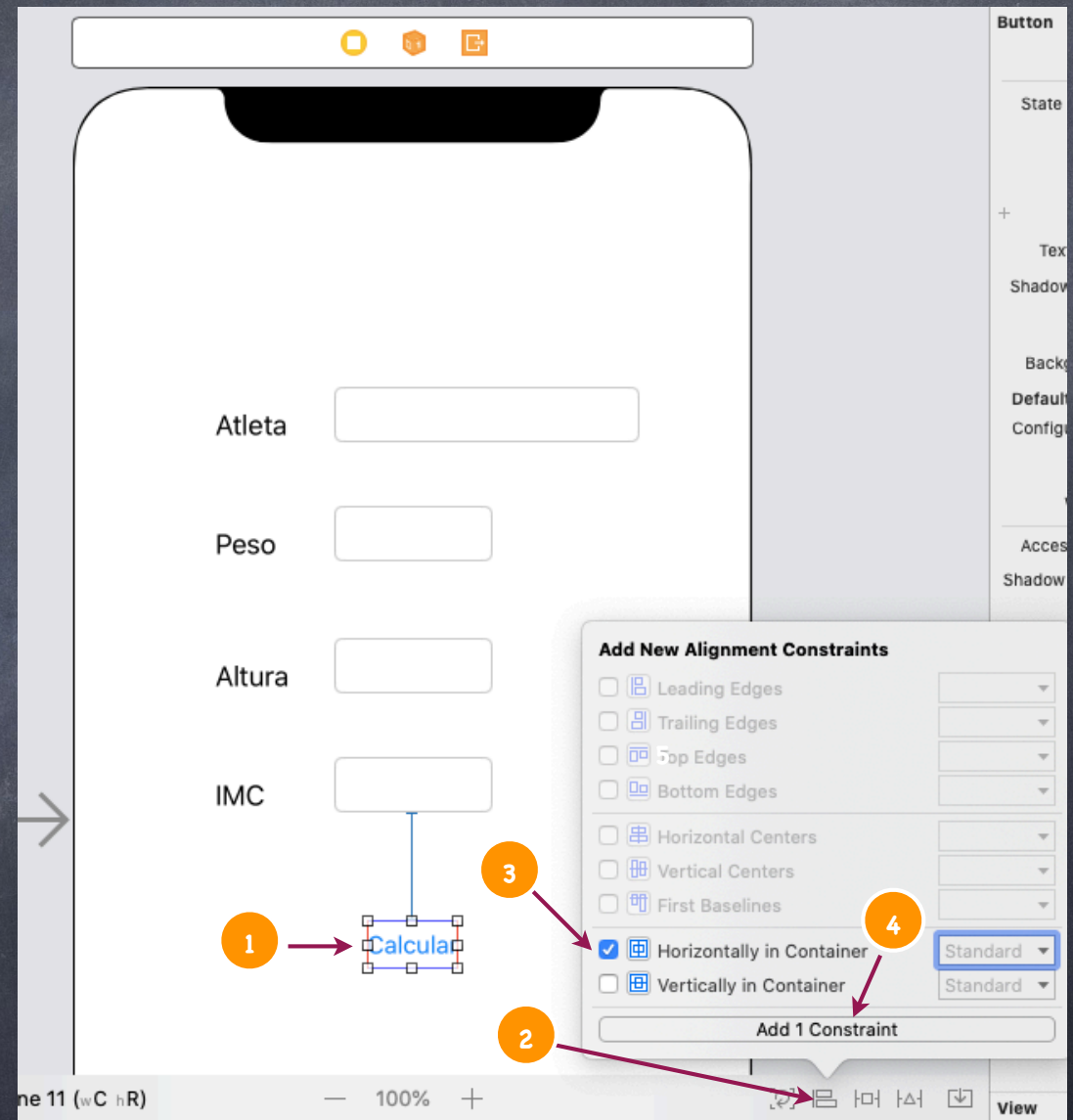
Auto Layout

- Por final falta criar a regra para o botão calcular, ele usará como base a distância do componente acima dele e com centralização horizontal, para isso selecione o Button (1), clique em Add New Constraints (2), marque a referência de topo (3), clique em Add 1 Constraint (4). Agora falta apenas o alinhamento horizontal que veremos no próximo slide.



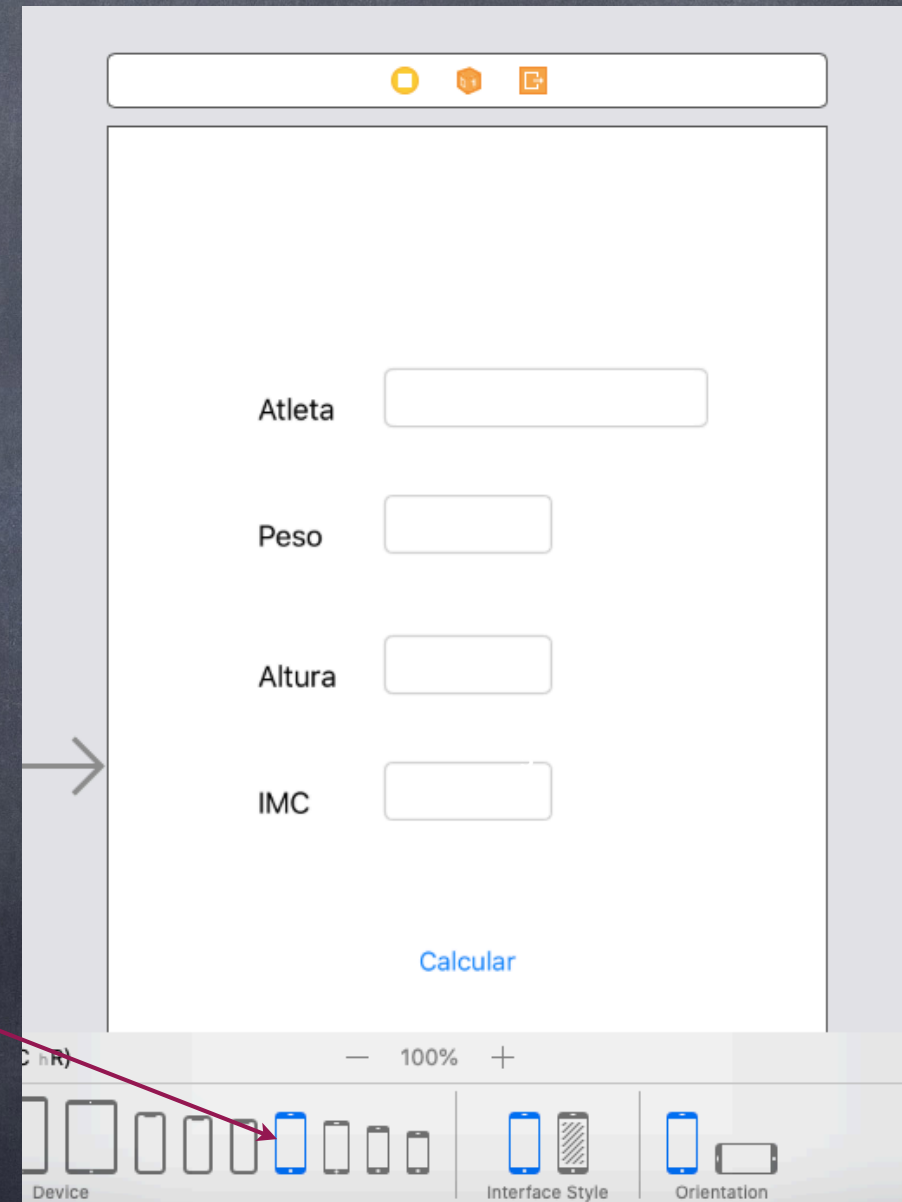
Auto Layout

- Para definir o alinhamento horizontal, selecione o Button (1), clique em Align (2), marque a opção Horizontally in Container (3), depois clique em Add 1 Constraint (4).



Auto Layout

- Pronto, agora basta clicar em qualquer device (1) e você irá observar que os componentes irão se adaptar aos vários tamanhos de iPhone e até Ipad. A tela ao lado representa os componentes em um Iphone 8 Plus (1). Mais detalhes de Auto Layout serão apresentados na aula 10, este é apenas um pequeno exemplo para que você se familiarize com essa facilidade que está presente desde o iOS 5 em nosso ambiente de desenvolvimento.



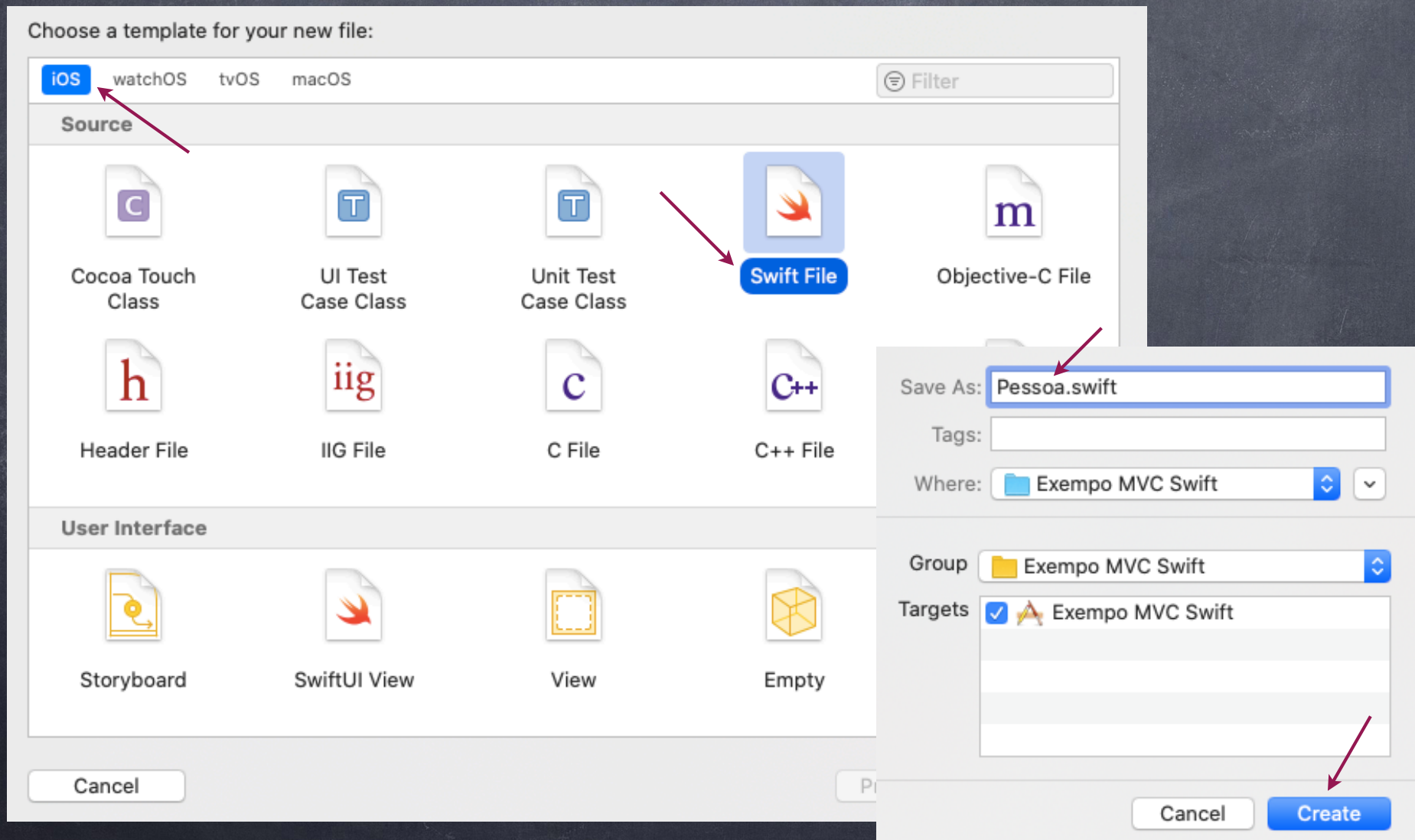
Outlet e Action

- No arquivo ViewController.swift, foi declarado como outlets todas as caixas de texto, além disso, foi criado um IBAction para o botão Calcular, ele irá disparar via TouchUpInside o método que está na model que nesse caso é a classe Pessoa.

```
1 //
2 // ViewController.swift
3 // Exemplo MVC Swift
4 //
5 // Created by Agesandro Scarpioni on 08/03/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights
  reserved.
7 //
8
9 import UIKit
10
11 class ViewController: UIViewController {
12
13     @IBOutlet weak var txtAtleta: UITextField!
14     @IBOutlet weak var txtPeso: UITextField!
15     @IBOutlet weak var txtAltura: UITextField!
16     @IBOutlet weak var txtImc: UITextField!
17
18     @IBAction func calcular(_ sender: Any) {
19
20     }
21
22     override func viewDidLoad() {
23         super.viewDidLoad()
24         // Do any additional setup after loading the view.
25     }
26
27 }
28
```

Criar a Model

- Crie uma classe (Command + N) em Swift chamada Pessoa.



Classe Pessoa

- Escreva class Pessoa { } e prepare-se para implementar a classe.

```
1 //  
2 // Pessoa.swift  
3 // Exempo MVC Swift  
4 //  
5 // Created by Agesandro Scarpioni on 08/03/20.  
6 // Copyright © 2020 Agesandro Scarpioni. All rights reserved.  
7 //  
8  
9 import Foundation  
10  
11 class Pessoa{  
12  
13 }  
14
```

- Nessa classe foi criado os atributos e métodos de Pessoa. Criei dois métodos que fazem a mesma coisa, porém, um método é do tipo "Void", ou seja, não tem retorno, e outro do tipo Float que retorna o valor calculado para quem o chamar.

```
1 //
2 // Pessoa.swift
3 // Exemplo MVC Swift
4 //
5 // Created by Agesandro Scarpioni on 08/03/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights reserved.
7 //
8
9 import Foundation
10
11 class Pessoa{
12     var nome:String!
13     var peso:Float!
14     var altura:Float!
15     var imc:Float!
16
17     //um exemplo sem parâmetro do tipo void
18     func calcular(){
19         imc = peso/(altura*altura)
20     }
21     //um exemplo sem parâmetro com retorno de um float
22     func calcular_2()->Float{
23         return peso/(altura*altura)
24     }
25 }
```

Obs: Em Swift uma classe que tem retorno e a que não tem retorno, ambas são declaradas como func, as que tem return são as funções e as que não possuem return são como o void do Java.

Construtores

- Para ilustrar foi criado na classe um construtor padrão e outro com 3 parâmetros, os get/sets foram criados com "!", ou seja, do tipo Optional que não precisa fazer o Unwrap, dessa forma os atributos podem receber null, por isso o construtor padrão não tem código. Dúvidas sobre Optional? Leia o Playground na tópico de Opcionais.

```
1 //
2 // Pessoa.swift
3 // Exemplo MVC Swift
4 //
5 // Created by Agesandro Scarpioni on 08/03/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights reserved.
7 //
8
9 import Foundation
10
11 class Pessoa{
12     var nome:String!
13     var peso:Float!
14     var altura:Float!
15     var imc:Float!
16
17     //um exemplo sem parâmetro do tipo void
18     func calcular(){
19         imc = peso/(altura*altura)
20     }
21     //um exemplo sem parâmetro com retorno de um float
22     func calcular_2()->Float{
23         return peso/(altura*altura)
24     }
25     //um exemplo com construtor padrão
26     init(){
27
28     }
29     //um exemplo de construtor com 3 parâmetros
30     init(nome:String, peso:Float, altura:Float){
31         self.peso = peso
32         self.altura = altura
33         self.nome = nome
34     }
35 }
```

Criando um objeto local

- Vamos começar com a criação do objeto "p", ou seja, uma instância da classe Pessoa (linha 19), usamos o construtor padrão, sem parâmetros, abaixo outras formas de instanciar o objeto (linhas 22 e 25).

```
1 //
2 // ViewController.swift
3 // Exemplo MVC Swift
4 //
5 // Created by Agesandro Scarpioni on 08/03/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights reserved.
7 //
8
9 import UIKit
10
11 class ViewController: UIViewController {
12
13     @IBOutlet weak var txtAtleta: UITextField!
14     @IBOutlet weak var txtPeso: UITextField!
15     @IBOutlet weak var txtAltura: UITextField!
16     @IBOutlet weak var txtImc: UITextField!
17
18     @IBAction func calcular(_ sender: Any) {
19         let p = Pessoa()
20
21         //Existem outras formas de criar o objeto p
22         //let p:Pessoa = Pessoa()
23
24         //ou com o construtor
25         //let p:Pessoa = Pessoa(nome: txtAtleta.text!, peso: Float(txtPeso.text!)! , altura:
26             Float(txtAltura.text!)! )
27
28         p.nome = txtAtleta.text
29     }
30
31     override func viewDidLoad() {
32         super.viewDidLoad()
33         // Do any additional setup after loading the view.
34     }
35 }
```

Criando um objeto público

- Se você tiver vários actions e todos eles precisem utilizar os conteúdos dos atributos ou funções do objeto "p", você precisará criar o objeto fora das funções e dentro da classe, dessa forma você terá acesso aos atributos e métodos do objeto de qualquer lugar do código. Veja a linha 18 com a criação do objeto "p" fora do action e seu uso na linha 21 dentro de um action.

```
1 //
2 // ViewController.swift
3 // Exemplo MVC Swift
4 //
5 // Created by Agesandro Scarpioni on 08/03/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights reserved.
7 //
8
9 import UIKit
10
11 class ViewController: UIViewController {
12
13     @IBOutlet weak var txtAtleta: UITextField!
14     @IBOutlet weak var txtPeso: UITextField!
15     @IBOutlet weak var txtAltura: UITextField!
16     @IBOutlet weak var txtImc: UITextField!
17
18     let p = Pessoa()
19
20     @IBAction func calcular(_ sender: Any) {
21         p.nome = txtAtleta.text
22     }
23
24     override func viewDidLoad() {
25         super.viewDidLoad()
26         // Do any additional setup after loading the view.
27     }
28
29 }
```

Programando a ViewController

- Optamos por criar um objeto local na linha 19 já que o único botão irá acessar todos os atributos e métodos da Classe, na linha 20 até a 22 passamos os dados da tela (view) para os atributos do objeto "p" da classe Pessoa (model). Na linha 24 foi chamado o método do tipo void, na linha 25 é exibido o conteúdo de imc do objeto "p" na caixa txtImc da View.

```
9  import UIKit
10
11  class ViewController: UIViewController {
12
13      @IBOutlet weak var txtAtleta: UITextField!
14      @IBOutlet weak var txtPeso: UITextField!
15      @IBOutlet weak var txtAltura: UITextField!
16      @IBOutlet weak var txtImc: UITextField!
17
18      @IBAction func calcular(_ sender: Any) {
19          let p = Pessoa()
20          p.nome = txtAtleta.text
21          p.peso = Float(txtPeso.text!)
22          p.altura = Float(txtAltura.text!)
23          //calculando o IMC pelo void que atualiza o atributo imc
24          p.calcular()
25          txtImc.text = "\(p.imc!)"
26      }
27
28      override func viewDidLoad() {
29          super.viewDidLoad()
30          // Do any additional setup after loading the view.
31      }
32
33 }
```

- Abaixo o calcular nas duas formas void (24 e 25) e a que retorna o Float(29), escolha uma das formas.

```
23      //calculando o IMC pelo void que atualiza o atributo imc
24      p.calcular()
25      txtImc.text = "\\(p.imc!)"
26
27      //Calculando o IMC pela função que retorna um Float,
28      //você utiliza uma linha a menos.
29      txtImc.text = "\\(p.calcular_2())"
```

- Quer controlar a exibição do resultado com apenas duas casas decimais ? Veja os exemplos abaixo:

```
23      //calculando o IMC pelo void que atualiza o atributo imc
24      p.calcular()
25      txtImc.text = String(format: "%.2f", p.imc)
26
27      //Calculando o IMC pela função que retorna um Float,
28      //você utiliza uma linha a menos.
29      txtImc.text = String(format: "%.2f", p.calcular_2())
```

Obs: Para converter um resultado float ou inteiro em String além da forma "\\(número)", use também String(número).

Validando os campos – parte 1

- Exemplo completo do código do IBAction validando os campos de peso e altura que são determinantes para o cálculo.

```
○ @IBAction func calcular(_ sender: Any) {
23
24     // Remove espaços extras colocando o conteúdo dentro de duas constantes
25     let pesoTexto = txtPeso.text?.trimmingCharacters(in: .whitespacesAndNewlines) ?? ""
26     let alturaTexto = txtAltura.text?.trimmingCharacters(in: .whitespacesAndNewlines) ?? ""
27
28     // Verifica se os campos estão vazios
29     if pesoTexto.isEmpty || alturaTexto.isEmpty {
30         mostrarAlerta(mensagem: "Preencha os campos de peso e altura.")
31         return
32     }
33
34     // Troca vírgula por ponto, caso o usuário digite 70,5
35     let pesoFormatado = pesoTexto.replacingOccurrences(of: ",", with: ".")
36     let alturaFormatada = alturaTexto.replacingOccurrences(of: ",", with: ".")
37
38     // Verifica se os valores são numéricos
39     guard let peso = Float(pesoFormatado),
40           let altura = Float(alturaFormatada) else {
41         mostrarAlerta(mensagem: "Digite apenas números válidos em peso e altura.")
42         return
43     }
44
45     // Verifica se os valores são maiores que zero
46     if peso <= 0 || altura <= 0 {
47         mostrarAlerta(mensagem: "Peso e altura devem ser maiores que zero.")
48         return
49     }
50 }
```

Fazendo o cálculo - parte 2

- Após a validação, as constantes serão levadas para os respectivos atributos do objeto, será chamado o método do tipo (void) na linha 63 e o resultado formatado para duas casas decimais na linha 67

```
50
51      // Cálculo do IMC
52      // criando o objeto p local
53      let p=Pessoa()
54      //passando o nome digitado para o atributo nome,
55      //não é importante para o cálculo
56      p.nome = txtNome.text
57      //passando as constantes peso e a altura já formatados
58      //e validados para os respectivos atributos do objeto p
59      p.peso = peso
60      p.altura = altura
61
62      //chamando o método (void) que atualiza o atributo IMC
63      p.calcular()
64
65      //formatando os dados para duas casas decimais,
66      //convertendo para string antes de levar para a tela (outlet)
67      txtImc.text = String(format: "%.2f", p.imc)
68  }
```

Preparando a mensagem – parte 3

- Fora das chaves do Action do botão calcular, crie uma nova função como na imagem abaixo para receber todas as mensagens da validação dos campos.

```
69
70 //criando uma função para receber o texto e mostrar a mensagem
71 func mostrarAlerta(mensagem: String) {
72     let alerta = UIAlertController(title: "Atenção", message: mensagem, preferredStyle: .alert)
73     alerta.addAction(UIAlertAction(title: "OK", style: .default))
74     present(alerta, animated: true)
75 }
```

- Execute seu código (Command + R) nesse exemplo de tela, foi digitado os dados na View que foram passados para a Model via Controller e a mesma recebeu o cálculo da model e atualizou o IMC na View, fechando o ciclo do MVC.
- Observe que a tela já está com Auto Layout.
- Pense na possibilidade de trocar a cor de fundo para aumentar o contraste entre o Text Field e o fundo da tela.

8:54

Atleta João

Peso 100

Altura 1.94

IMC 26.57

Calcular

Criação de um programa para testarmos todos os conceitos desta aula.

- Crie um projeto para Restaurante com uma Classe Cliente que possua seus atributos e métodos, um exemplo de método que você possa implementar é PagarConta que retorna o cálculo do troco quando o valor for recebido em dinheiro, esse método precisa receber o valor da conta e o valor pago.
- Um método ReservarMesa que tenha como parâmetro inteiro de entrada a hora cheia e retorne a palavra "almoço" ou "jantar" dependendo do horário.
- Exemplo: Entre 11 e 15 exiba "almoço", entre 18 e 23 exiba "jantar", caso contrário exiba "horário indisponível para reserva.
- Valide os campos.